

# CMSC 25300 / 35300 (Spring 2024)

## Homework 6

### Submission Instructions:

- Please submit your homework in PDF to Gradescope (which can be accessed from the course's website on Canvas);
- Please paste your code in the submitted PDF. In other words, your submission should be a single PDF that contains both your writing solutions and your code. Be sure to include the output of your code when relevant (e.g. texts and figures). Keep in mind that we are not easily able to run your code.
- Note that you do not need to copy the problem statements in your solution, *as long as you clearly indicate the problem numbers* (e.g., 1.a, 2.c, etc).

### 1. Kernel Ridge Regression.

- a) (10 pts) In kernel ridge regression (KRR), we convert the data matrix  $\mathbf{X} \in \mathbb{R}^{n \times p}$  to  $\Phi \in \mathbb{R}^{n \times d}$  by running each data  $x_i$  through the nonlinear transformation function  $\phi$ . Correspondingly, the goal in KRR is to minimize find a weight vector that minimizes the squared loss and the regularization term:

$$\hat{w}_{KRR} = \arg \min_{w \in \mathbb{R}^d} \|\Phi w - y\|_2^2 + \lambda \|w\|_2^2$$

Derive a closed form formula for  $\hat{w}_{KRR}$  in terms of (possibly a subset of)  $I, \lambda, \Phi, y$ . Your derivations should not make any assumptions about the rank of  $\Phi$ .

(Hint: need to explain why your derivation or solution is valid even without any assumptions about the rank of  $\Phi$ ).

- b) (15 pts) Recall (from Lecture 11, 12 and Recitation 6) that the weights vector  $\hat{w}_{KRR}$  is a linear combination of training samples,  $\hat{w}_{KRR} = \Phi^T \alpha$ , where  $\alpha$  is a vector containing the coefficients. Additionally, the kernel function  $K$  can be used to construct a matrix  $\mathbf{K}$ , where  $\mathbf{K}_{ij} = K(x_i, x_j)$ .

- i) (7 pts) Assume the rows of  $\Phi$  are linearly independent (which is often true when  $d$  is very large). Use your answer from part (a) to derive an expression for  $\alpha$  in terms of  $\mathbf{K}, \lambda, I, y$ .

- ii) (8 pts) Now we want to make prediction on a new example  $z$ , we know that  $\hat{y} = w_{KRR}^T \phi(z)$ . Rewrite this expression using  $\alpha$  and then substitute  $\alpha$  with the expression you found in (i). Simplify the expression such that the final version only

uses  $\mathbf{K}, \lambda, I, y, X, z$ . (Hint: you may find the notation  $K(\mathbf{X}, z) = \begin{bmatrix} K(x_1, z) \\ K(x_2, z) \\ \dots \\ K(x_n, z) \end{bmatrix}$

useful.)

c) (10 pts) Explain the relationship between  $\phi$  and  $K$ , and between  $\Phi$  and  $\mathbf{K}$  mathematically. And then explain why the relationship is significant/useful. (Hint: Why do we have these two functions / matrices? What benefit do we get by having the kernel function  $K$  in kernel methods?)

d) (20 pts) Implement Kernel Ridge regression.

Generate training samples using the following code:

```
n = 200;
p = 2;
X = 2*(np.random.rand(n,p) - .5)
y = np.sign(X[:,1] - (X[:,0]**2/2 + np.sin(X[:,0]*7)/2))

plt.figure(1)
plt.scatter(X[:, 0], X[:, 1], 50, c=y)
plt.colorbar()
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('2d_training_samples_colored_by_label')
plt.show()
```

Then train a kernel ridge regression classifier using a Gaussian kernel (recall Gaussian kernel  $K(x_i, x_j) = e^{-\frac{\|x_i - x_j\|_2^2}{2\sigma^2}}$ ). I suggest starting with  $\lambda = 1$  and  $\sigma^2 = .05$ . Then vary  $\sigma^2$ ; how does this affect your classifier? The following code may be helpful:

```
sigma = .05
lam = 1

### YOUR CODE STARTS HERE ###
# K = ...
# alpha = ...
# yhat = ...
### YOUR CODE ENDS HERE ###

y2 = np.array(np.sign(yhat))
plt.figure(2)
plt.scatter(X[:, 0], X[:, 1], 50, c=y2)
plt.colorbar()
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('2d_training_samples_colored_by_PREDICTED_label')
plt.show()

ntest = 2000;
Xtest = 2*(np.random.rand(ntest,p) - .5)
```

```

### YOUR CODE STARTS HERE ###
# K_test = ...
# ytest = ...
### YOUR CODE ENDS HERE ###

plt.figure(3)
plt.scatter(Xtest[:,0], Xtest[:,1], 50, c=np.array(ytest))
plt.colorbar()
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('2d_test_samples_colored_by_PREDICTED_label(
    before_taking_sign)')
plt.show()

```

**2. Support Vector Machines.** When training a support vector machine, we are given  $n$  pairs of training data samples and labels:  $\{\mathbf{x}_i, y_i\}_{1 \leq i \leq n}$ , where  $\mathbf{x}_i \in \mathbb{R}^p$  and labels  $y_i \in \{1, -1\}$ . If we are using a kernel function  $K(\mathbf{x}_i, \mathbf{x}_j)$ , then a first step is to create the kernel matrix  $\mathbf{K}$ , where  $\mathbf{K}_{i,j} = K(\mathbf{x}_i, \mathbf{x}_j)$ . In the rest of this problem, we will use  $\mathbf{k}_i$  for the  $i^{\text{th}}$  row of  $\mathbf{K}$ . To train the SVM, we are trying to solve this optimization problem:

$$\arg \min_{\boldsymbol{\alpha}} f(\boldsymbol{\alpha}) = \arg \min_{\boldsymbol{\alpha}} \sum_{i=1}^n (1 - y_i \mathbf{k}_i^T \boldsymbol{\alpha})_+ + \lambda \boldsymbol{\alpha}^T \mathbf{K} \boldsymbol{\alpha} \quad (1)$$

In the following problems, we will make two standard assumptions about  $K(\mathbf{x}_i, \mathbf{x}_j)$ :

- For any set of samples  $\{\mathbf{x}_i\}_{1 \leq i \leq n}$ ,  $\mathbf{K}$  is a positive semidefinite matrix. In this case, we call  $K(\mathbf{x}_i, \mathbf{x}_j)$  a *positive semidefinite kernel*.
  - $K(\mathbf{x}_i, \mathbf{x}_j) = K(\mathbf{x}_j, \mathbf{x}_i)$ . In this case, we call  $K(\mathbf{x}_i, \mathbf{x}_j)$  a *symmetric kernel*. This means the matrix  $\mathbf{K}$  is also symmetric (so its  $i^{\text{th}}$  column is also the  $\mathbf{k}_i$  vector).
- a) We will eventually want to discuss training an SVM by optimizing equation (1) using gradient descent. We know that gradient descent works well when optimizing *convex* functions. Recall that a function  $g(x)$  is convex if for every  $w, v \in \mathbb{R}^p$ ,

$$g(w) \geq g(v) + \nabla g(v)^T (w - v)$$

One way to think about this definition of a convex function: a function is convex when its first-order Taylor approximation is always less than or equal to the true function value. You already had the geometric intuition from class that  $f$  is convex, but in this problem you will formally prove  $f$  in Equation (1) is convex.

- i) (10 points) First, suppose that  $g(x) = g_1(x) + g_2(x)$ , where  $g_1(x)$  and  $g_2(x)$  are both convex functions. Show that  $g(x)$  is also a convex function.

- ii) (10 points) In this problem, we will show that  $g(\mathbf{w}) = \mathbf{w}^T \mathbf{K} \mathbf{w}$  is a convex function of  $\mathbf{w}$ . To prove  $g(\mathbf{w})$  is convex, we can write out its Taylor expansion, centered at a new point  $\mathbf{v}$ . Because  $g$  is a quadratic function, there are only three terms in the Taylor expansion:

$$\begin{aligned} g(\mathbf{w}) &= g(\mathbf{v}) + \nabla_{\mathbf{v}} g^T (\mathbf{w} - \mathbf{v}) + \frac{1}{2} (\mathbf{w} - \mathbf{v})^T \nabla_{\mathbf{v}}^2 g (\mathbf{w} - \mathbf{v}) \\ &= \mathbf{v}^T \mathbf{K} \mathbf{v} + 2(\mathbf{K} \mathbf{v})^T (\mathbf{w} - \mathbf{v}) + (\mathbf{w} - \mathbf{v})^T \mathbf{K} (\mathbf{w} - \mathbf{v}) \end{aligned}$$

Looking at this expression, which part is the first-order Taylor approximation? How do we know that  $g$  is convex?

- b) (10 points) In the previous problem, we convinced ourselves that gradient descent is an appropriate choice for minimizing Equation (1). This is because we found that  $f$  is a sum of convex functions, so  $f$  itself is also convex. Now we will think about minimizing equation 1 using gradient descent with step size  $\tau$ . Suppose we choose a random initialization for  $\boldsymbol{\alpha}^{(1)}$ , and after  $t$  steps, all of the points are correctly classified with margin  $> 1$  (formally,  $y_i \mathbf{k}_i^T \boldsymbol{\alpha} > 1$ ). In this case, what is the value of  $f(\boldsymbol{\alpha}^{(t)})$ ? What is the value of  $\boldsymbol{\alpha}^{(t+1)}$  (as a function of  $\boldsymbol{\alpha}^{(t)}$  and other parameters)?
- c) (15 points) In the previous problem, some of the terms in the loss function  $f$  were zero at  $\boldsymbol{\alpha}^{(t)}$ . If we keep iterating gradient descent and find  $\boldsymbol{\alpha}^{(t+1)}, \boldsymbol{\alpha}^{(t+2)}, \boldsymbol{\alpha}^{(t+3)}, \boldsymbol{\alpha}^{(t+4)}, \dots$  will this remain true? Is this true at the optimal value  $\boldsymbol{\alpha}^*$ ? To answer this, remember that  $\nabla_{\boldsymbol{\alpha}} f|_{\boldsymbol{\alpha}=\boldsymbol{\alpha}^*} = 0$ . Expand this equation using the formula for  $\nabla_{\boldsymbol{\alpha}} f$ , and explain what this tells us about the optimal solution.